Analytical Problem Solving

Warehouse Robot Navigation using BFS and DFS

SOLUTION:

Problem Set-up and Assumptions

The warehouse is a 5×5 grid. Cells are written as (row, column), numbered 1 to 5 from the top-left. The robot starts at (1,1) and must reach (5,5). Blocked shelving cells are B = {(2,2), (2,3), (3,3), (4,2), (4,4)}, so there are 25 − 5 = **20 free cells**.

r \ c	**1**	**2**	**3**	**4**	**5**
1	**START (1,1)**	(1,2)	(1,3)	(1,4)	(1,5)
2	(2,1)	**X**	**X**	(2,4)	(2,5)
3	(3,1)	(3,2)	**X**	(3,4)	(3,5)
4	(4,1)	**X**	(4,3)	**X**	(4,5)
5	(5,1)	(5,2)	(5,3)	(5,4)	**GOAL (5,5)**

- **Grid key:** X = blocked shelf cell.
- **Moves:** Up = (r−1, c), Down = (r+1, c), Left = (r, c−1), Right = (r, c+1). No diagonal moves; every move costs 1.
- **Tie-breaking:** neighbours are always considered in the order **Up, Down, Left, Right**, so the traces are reproducible.
- **Graph search:** both BFS and DFS keep a visited/explored set, so no state is expanded twice.
- **Goal test:** BFS tests when a node is generated; DFS tests when a node is popped (expanded).
- The robot has the package loaded at the start, and the warehouse map is known.

Part (a): PEAS Description of the Warehouse Robot

PEAS	**Description for this warehouse robot**
Performance measure	Reach the goal cell (5,5) with the package intact (delivery success). Minimise the path cost (number of moves). The optimum here is 8 moves.

	No collisions with shelving units; no unnecessary or repeated moves. Minimise time and battery energy used.
Environment	A 5×5 grid warehouse of 25 cells: 20 free cells and 5 fixed shelving cells (blocked). Start cell (1,1), goal cell (5,5), only four-directional movement. **Properties:** fully observable (map known), single-agent, deterministic, static (shelves never move), discrete, sequential, known.
Actuators	Drive motors / wheels to move one cell Up, Down, Left or Right. Steering or turning mechanism and brakes to stop in each cell. Package-handling mechanism (gripper or tray) to release the package at (5,5).
Sensors	Position sensors (wheel encoders, floor QR/RFID tags) to know the current cell. Obstacle sensors (LIDAR, ultrasonic, bump sensors) to detect shelving units. Camera to recognise the goal marker, and a package-presence sensor. Battery-level sensor.

(b): Agent Type Classification

Best choice: a goal-based agent (one that keeps a model of the grid and plans a sequence of actions to reach a goal).

Agent type	**Suitable or not**	**Reasoning specific to this scenario**
Simple reflex	No	Acts only on the current percept, with no memory or lookahead. The grid has dead-end pockets at (3,2) and (4,3) and walls such as (2,2)-(2,3). A rule like "move Right/Down toward the goal" fails at (1,2) (Down is blocked) and can loop forever or enter a dead end.
Model-based reflex	Partly	Tracks its position and the map, which helps avoid loops, but it still only applies condition-action rules. It has no notion of the destination, so it cannot choose a route.
Goal-based	**Yes**	The task has one explicit goal state, (5,5). The agent must ask "which sequence of moves leads to the goal?", which is exactly search. The environment is deterministic, static and

		fully observable, so it can plan the whole path offline and then execute it.
Utility-based	Not needed	Useful when goals conflict or outcomes differ in quality (speed vs safety vs battery). Here there is a single objective and every step costs 1, so path cost already captures what matters. It would become relevant if cells had different costs or risks.

Justification summary: the shelves create dead ends, so the robot must look ahead rather than react. The goal is well defined and checkable, and the uniform step cost means that finding a goal-reaching (and ideally shortest) action sequence is the whole problem. A goal-based agent with a grid model and a search algorithm fits this exactly.

(c): Formal Search Problem Formulation

Component	**Definition**
State space	$S = \{ (r, c) : 1 \leq r, c \leq 5$ and $(r, c) \notin B \}$, which has 20 states.
Initial state	$s_0 = (1,1)$
Goal state / test	Goal = (5,5). GoalTest(s) is true iff s = (5,5).
Actions	ACTIONS(r, c) = the subset of {Up, Down, Left, Right} whose resulting cell lies inside the grid (1 to 5) and is not in B. Example: ACTIONS(1,1) = {Down, Right}; ACTIONS(3,2) = {Left}.
Transition model	RESULT((r,c), Up) = (r−1, c) RESULT((r,c), Down) = (r+1, c) RESULT((r,c), Left) = (r, c−1) RESULT((r,c), Right) = (r, c+1) (each defined only when the result is a legal free cell)
Path cost	Step cost $c(s, a, s') = 1$ for every action. Path cost g(n) = sum of step costs = number of moves in the path.
Solution	A sequence of actions from (1,1) to (5,5). It is optimal if its cost is minimal. The Manhattan lower bound is $\|5-1\| + \|5-1\| = 8$, so the optimal cost is 8.

(d): Breadth-First Search (BFS)

BFS uses a FIFO queue and expands the shallowest node first. The queue starts as [(1,1)]. The "Queue after" column lists the queue from front to back after each expansion.

Step	**Expand**	**New nodes added (U, D, L, R order)**	**Queue after step (front → back)**
1	(1,1)	(2,1), (1,2)	(2,1), (1,2)
2	(2,1)	(3,1)	(1,2), (3,1)
3	(1,2)	(1,3)	(3,1), (1,3)
4	(3,1)	(4,1), (3,2)	(1,3), (4,1), (3,2)
5	(1,3)	(1,4)	(4,1), (3,2), (1,4)
6	(4,1)	(5,1)	(3,2), (1,4), (5,1)
7	(3,2)	none (dead end)	(1,4), (5,1)
8	(1,4)	(2,4), (1,5)	(5,1), (2,4), (1,5)
9	(5,1)	(5,2)	(2,4), (1,5), (5,2)
10	(2,4)	(3,4), (2,5)	(1,5), (5,2), (3,4), (2,5)
11	(1,5)	none ((2,5) already in queue)	(5,2), (3,4), (2,5)
12	(5,2)	(5,3)	(3,4), (2,5), (5,3)
13	(3,4)	(3,5)	(2,5), (5,3), (3,5)
14	(2,5)	none ((3,5) already in queue)	(5,3), (3,5)
15	(5,3)	(4,3), (5,4)	(3,5), (4,3), (5,4)
16	(3,5)	(4,5)	(4,3), (5,4), (4,5)
17	(4,3)	none (dead end)	(5,4), (4,5)
18	(5,4)	**(5,5) = GOAL generated**	(4,5), (5,5)

Path reconstruction (via parent pointers): (5,5) ← (5,4) ← (5,3) ← (5,2) ← (5,1) ← (4,1) ← (3,1) ← (2,1) ← (1,1).

Final path: (1,1) → (2,1) → (3,1) → (4,1) → (5,1) → (5,2) → (5,3) → (5,4) → (5,5)

Actions: Down, Down, Down, Down, Right, Right, Right, Right. **Total cost = 8.**

- 18 nodes were expanded and all 20 free states were generated. The largest queue size was 4 (after step 10).
- The cost of 8 equals the Manhattan lower bound, so the path is optimal. Other cost-8 paths exist (for example along the top row and right column), but this tie-breaking order finds the bottom-left route first.

- If the goal test were applied on expansion instead of on generation, (5,5) would be popped at step 20 with the same path and cost.

(e): Depth-First Search (DFS)

DFS uses a LIFO stack. Neighbours are pushed in reverse order (Right, Left, Down, Up) so that Up is popped first. The stack starts as [(1,1)]. The "Stack after" column lists the stack from top to bottom after each step.

Step	**Pop / expand**	**Pushed (top first)**	**Stack after step (top → bottom)**
1	(1,1)	(2,1), (1,2)	(2,1), (1,2)
2	(2,1)	(3,1)	(3,1), (1,2)
3	(3,1)	(4,1), (3,2)	(4,1), (3,2), (1,2)
4	(4,1)	(5,1)	(5,1), (3,2), (1,2)
5	(5,1)	(5,2)	(5,2), (3,2), (1,2)
6	(5,2)	(5,3)	(5,3), (3,2), (1,2)
7	(5,3)	(4,3), (5,4)	(4,3), (5,4), (3,2), (1,2)
8	(4,3)	none (dead end, backtrack)	(5,4), (3,2), (1,2)
9	(5,4)	(5,5)	(5,5), (3,2), (1,2)
10	(5,5)	**GOAL TEST PASSED**	(3,2), (1,2) (left unexplored)

Path reconstruction: (5,5) ← (5,4) ← (5,3) ← (5,2) ← (5,1) ← (4,1) ← (3,1) ← (2,1) ← (1,1).

Final path: (1,1) → (2,1) → (3,1) → (4,1) → (5,1) → (5,2) → (5,3) → (5,4) → (5,5)

Actions: Down, Down, Down, Down, Right, Right, Right, Right. **Total cost = 8.**

- 10 nodes were expanded (9 non-goal plus the goal). The largest stack size was 4 (after step 7). The branches (3,2) and (1,2) were never explored.
- The only backtrack was at the dead end (4,3) in step 8.
- DFS found an optimal path here only because of the Up, Down, Left, Right ordering. With the order Right, Left, Down, Up, DFS returns (1,1)→(1,2)→(1,3)→(1,4)→(1,5)→(2,5)→(2,4)→(3,4)→(3,5)→(4,5)→(5,5), which costs **10**.

(f): Comparison of BFS and DFS

Notation: b = branching factor (at most 4; at most 3 once the parent is excluded), d = depth of the shallowest goal (8), m = maximum path depth (at most 19, since there are 20 free cells), |V| = 20 states.

Criterion	BFS	DFS
Completeness	Complete: the state space is finite and the goal is at finite depth d = 8, so BFS always finds it.	Complete here only because of the visited set on a finite graph. Plain tree-search DFS can loop forever on this grid, since moves are reversible (Up/Down, Left/Right).
Optimality	Optimal: step costs are uniform, so the shallowest goal is the cheapest. It found cost 8, equal to the lower bound.	Not optimal in general. It found cost 8 here by luck of ordering; another order gives cost 10 (Part e).
Time complexity	O(b^d). Tree-search worst case is about $4^8 = 65{,}536$ nodes; with a visited set it is O(|V| + |E|). Observed: 18 expansions.	O(b^m). With a visited set it is also O(|V| + |E|). Observed: 10 expansions (fewer because the first branch led to the goal).
Space complexity	O(b^d): the whole frontier is stored. Observed maximum queue size 4, plus a visited set of up to 20 states.	O(b·m): only the current path and its unexplored siblings are stored (linear in depth). Observed maximum stack size 4, plus the visited set.

BFS is more suitable because a delivery robot cares about the number of moves, which means time and energy, and every move has the same cost. BFS guarantees the fewest moves, while DFS gives no such guarantee and its path quality depends on arbitrary neighbour ordering. The grid is tiny, so BFS's larger memory use is not a problem. DFS's advantage (10 vs 18 expansions) comes from lucky ordering rather than any guarantee. For a much larger warehouse, an informed method such as A with a Manhattan-distance heuristic would be a better choice still.

Appendix: Python Code and Output

PYTHON CODE

CO1 AT 1 CODE.txt - C:/Users/RUCHITA/Documents/AI LAB PROGRAMS/CO1 AT 1 CODE.txt (3.14.7)

File Edit Format Run Options Window Help

```
"""
Warehouse delivery robot: BFS and DFS on a 5x5 grid.
Cells are (row, col), 1-indexed. Up = row-1, Down = row+1, Left = col-1, Right = col+1.
Neighbour order everywhere: Up, Down, Left, Right.
"""
from collections import deque

N = 5
START, GOAL = (1, 1), (5, 5)
BLOCKED = {(2, 2), (2, 3), (3, 3), (4, 2), (4, 4)}

# Action name -> (d_row, d_col); dict order = expansion order
ACTIONS = {"Up": (-1, 0), "Down": (1, 0), "Left": (0, -1), "Right": (0, 1)}


def actions(state):
    """Return list of (action, next_state) that are legal from `state`."""
    r, c = state
    result = []
    for name, (dr, dc) in ACTIONS.items():
        nr, nc = r + dr, c + dc
        if 1 <= nr <= N and 1 <= nc <= N and (nr, nc) not in BLOCKED:
            result.append((name, (nr, nc)))
    return result

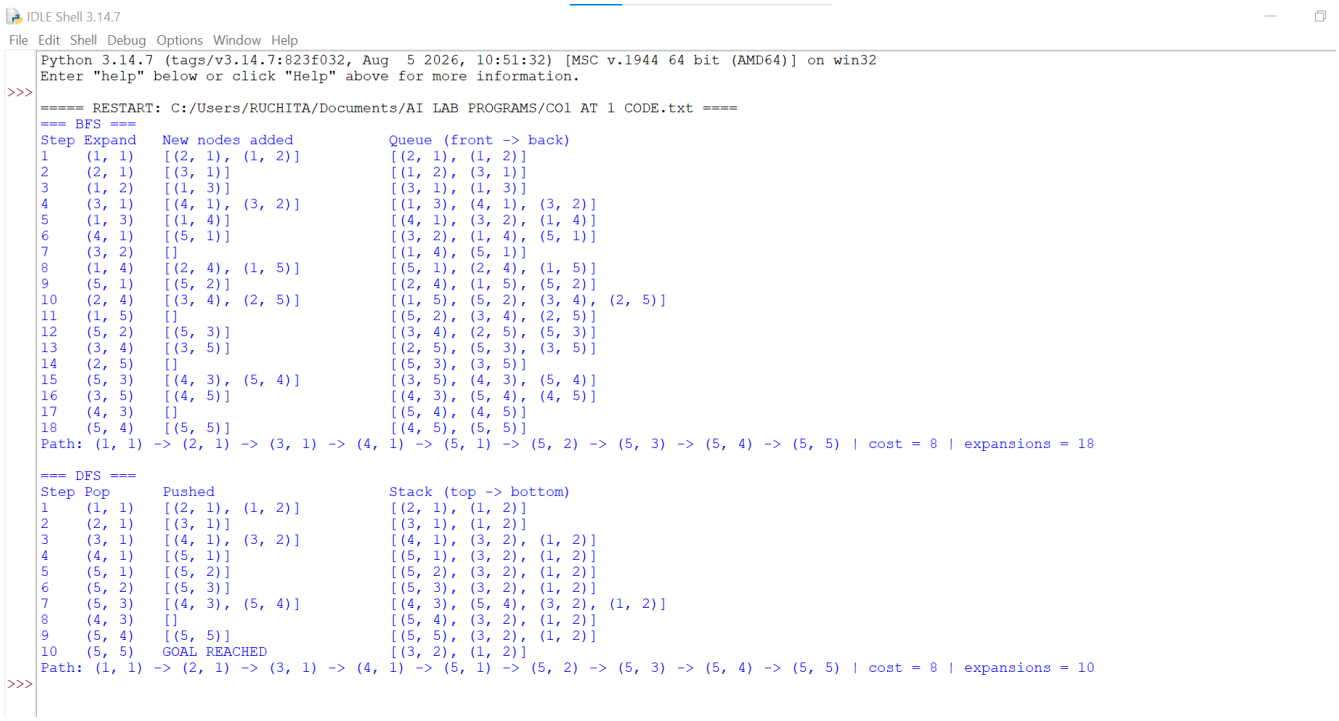

def build_path(parent, state):
    path = []
    while state is not None:
        path.append(state)
        state = parent[state]
    return path[::-1]


def bfs(start=START, goal=GOAL):
    frontier = deque([start])            # FIFO queue
    parent = {start: None}               # also serves as the "visited" set
    step = 0
    print(f"{'Step':<5}{'Expand':<9}{'New nodes added':<26}Queue (front -> back)")
    while frontier:
        node = frontier.popleft()
        step += 1

        if node == goal:                 # goal test at expansion time
            print(f"{step:<5}{str(node):<9}{'GOAL REACHED':<26}{stack[::-1]}")
            path = build_path(parent, goal)
            return path, len(path) - 1, step
        pushed = []
        # push in reverse so that Up is popped first, then Down, Left, Right
        for _, nxt in reversed(actions(node)):
            if nxt not in visited:
                parent[nxt] = node       # latest parent wins (standard DFS)
                stack.append(nxt)
                pushed.append(nxt)
        print(f"{step:<5}{str(node):<9}{str(pushed[::-1]):<26}{stack[::-1]}")
    return None, None, step


if __name__ == "__main__":
    print("=== BFS ===")
    p, cost, n = bfs()
    print("Path:", " -> ".join(map(str, p)), "| cost =", cost, "| expansions =", n)
    print("\n=== DFS ===")
    p, cost, n = dfs()
    print("Path:", " -> ".join(map(str, p)), "| cost =", cost, "| expansions =", n)
```

PROGRAM OUTPUT:



```
IDLE Shell 3.14.7
File  Edit  Shell  Debug  Options  Window  Help
Python 3.14.7 (tags/v3.14.7:823f032, Aug  5 2026, 10:51:32) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/RUCHITA/Documents/AI LAB PROGRAMS/CO1 AT 1 CODE.txt ====
=== BFS ===
Step Expand   New nodes added          Queue (front -> back)
1    (1, 1)   [(2, 1), (1, 2)]         [(2, 1), (1, 2)]
2    (2, 1)   [(3, 1)]                 [(1, 2), (3, 1)]
3    (1, 2)   [(1, 3)]                 [(3, 1), (1, 3)]
4    (3, 1)   [(4, 1), (3, 2)]         [(1, 3), (4, 1), (3, 2)]
5    (1, 3)   [(1, 4)]                 [(4, 1), (3, 2), (1, 4)]
6    (4, 1)   [(5, 1)]                 [(3, 2), (1, 4), (5, 1)]
7    (3, 2)   []                       [(1, 4), (5, 1)]
8    (1, 4)   [(2, 4), (1, 5)]         [(5, 1), (2, 4), (1, 5)]
9    (5, 1)   [(5, 2)]                 [(2, 4), (1, 5), (5, 2)]
10   (2, 4)   [(3, 4), (2, 5)]         [(1, 5), (5, 2), (3, 4), (2, 5)]
11   (1, 5)   []                       [(5, 2), (3, 4), (2, 5)]
12   (5, 2)   [(5, 3)]                 [(3, 4), (2, 5), (5, 3)]
13   (3, 4)   [(3, 5)]                 [(2, 5), (5, 3), (3, 5)]
14   (2, 5)   []                       [(5, 3), (3, 5)]
15   (5, 3)   [(4, 3), (5, 4)]         [(3, 5), (4, 3), (5, 4)]
16   (3, 5)   [(4, 5)]                 [(4, 3), (5, 4), (4, 5)]
17   (4, 3)   []                       [(5, 4), (4, 5)]
18   (5, 4)   [(5, 5)]                 [(4, 5), (5, 5)]
Path: (1, 1) -> (2, 1) -> (3, 1) -> (4, 1) -> (5, 1) -> (5, 2) -> (5, 3) -> (5, 4) -> (5, 5) | cost = 8 | expansions = 18

=== DFS ===
Step Pop      Pushed                  Stack (top -> bottom)
1    (1, 1)   [(2, 1), (1, 2)]        [(2, 1), (1, 2)]
2    (2, 1)   [(3, 1)]                [(3, 1), (1, 2)]
3    (3, 1)   [(4, 1), (3, 2)]        [(4, 1), (3, 2), (1, 2)]
4    (4, 1)   [(5, 1)]                [(5, 1), (3, 2), (1, 2)]
5    (5, 1)   [(5, 2)]                [(5, 2), (3, 2), (1, 2)]
6    (5, 2)   [(5, 3)]                [(5, 3), (3, 2), (1, 2)]
7    (5, 3)   [(4, 3), (5, 4)]        [(4, 3), (5, 4), (3, 2), (1, 2)]
8    (4, 3)   []                      [(5, 4), (3, 2), (1, 2)]
9    (5, 4)   [(5, 5)]                [(5, 5), (3, 2), (1, 2)]
10   (5, 5)   GOAL REACHED            [(3, 2), (1, 2)]
Path: (1, 1) -> (2, 1) -> (3, 1) -> (4, 1) -> (5, 1) -> (5, 2) -> (5, 3) -> (5, 4) -> (5, 5) | cost = 8 | expansions = 10
>>>
```

CONCLUSION:

The warehouse delivery robot problem was modelled as a goal-based intelligent agent. Its PEAS description was defined, and the task was formulated as a search problem with an initial state (1,1), a goal state (5,5), four movement actions, a transition model and a uniform step cost of 1.

Breadth-First Search and Depth-First Search were applied with the same neighbour order (Up, Down, Left, Right). Both reached the goal along the same path:

(1,1) → (2,1) → (3,1) → (4,1) → (5,1) → (5,2) → (5,3) → (5,4) → (5,5), with a **total cost of 8**.

This cost equals the Manhattan-distance lower bound, so the path is optimal. BFS expanded 18 nodes and DFS expanded 10. DFS expanded fewer nodes, but it is not guaranteed to be optimal: with the neighbour order Right, Left, Down, Up it returns a path of cost 10. BFS is complete and optimal for uniform step costs, so it is the more suitable algorithm for this warehouse. The grid is small, so its higher memory use is not a concern.

Overall, a goal-based agent using BFS gives the robot a reliable, shortest-path delivery route.